Empirical Analysis of Sorting Algorithms

Eman Gurung

November 2026

1 Experimental Setup

Three sorting algorithms were implemented in C++: Insertion Sort, Randomized Quicksort, and Counting Sort. The algorithms were tested using uniformly generated random integers in the range $[0, k]$, with input sizes $n \in \{10, 100, 1000, 2000, 5000, 10000, 20000\}$. For each input size, two key ranges were tested: $k = n$ and $k = 100n$.

Each configuration was evaluated over five independent trials. All algorithms received identical copies of the input during each trial. Execution times were measured using `std::chrono::steady_clock`, and the median runtime was recorded in milliseconds.

2 Experimental Results

n	k	Insertion (ms)	Quicksort (ms)	Counting (ms)
10	10	0.001464	0.002929	0.009469
10	1000	0.001464	0.002928	0.097522
100	100	0.042269	0.026455	0.029774
100	10000	0.037291	0.026846	0.782130
1000	1000	1.625960	0.199144	0.124075
1000	100000	1.475130	0.208027	3.865160
2000	2000	5.557110	0.411468	0.244538
2000	200000	5.554860	0.409905	7.640900
5000	5000	32.779100	1.079680	0.534176
5000	500000	33.617200	1.066110	18.973700
10000	10000	132.694000	2.163160	0.990549
10000	1000000	133.566000	2.193220	38.838400
20000	20000	523.495000	4.579350	1.924970
20000	2000000	541.609000	4.511020	78.618700

Table 1: Median runtime over five trials, measured in milliseconds.

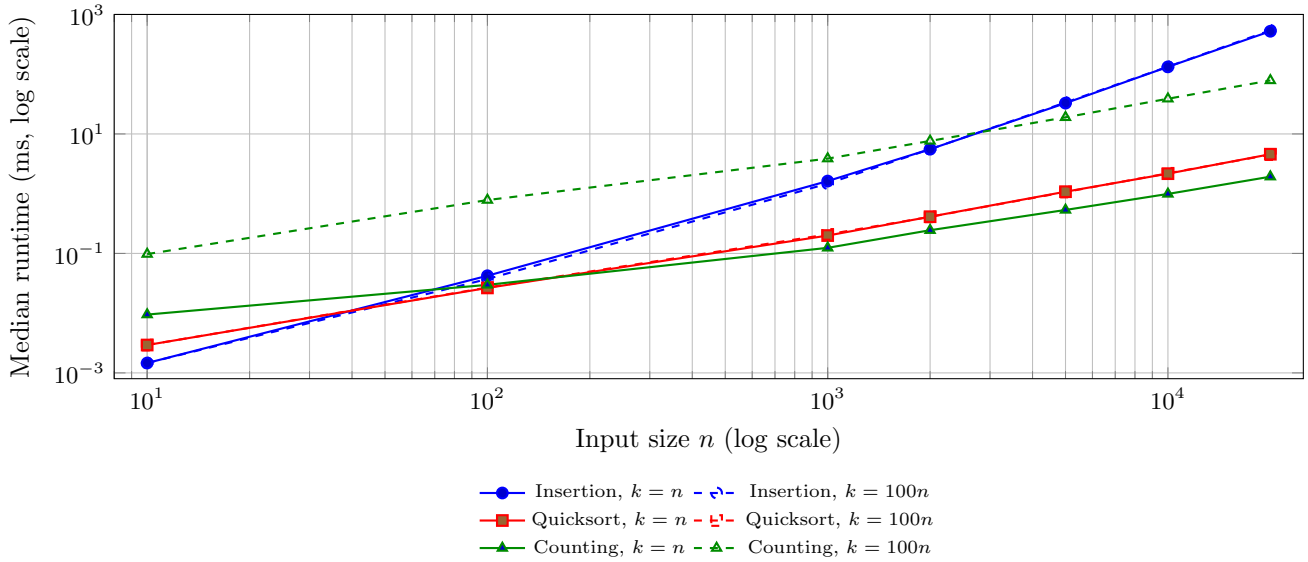


Figure 1: Median runtimes of the three algorithms at two key ranges. Both axes are logarithmic.

3 Discussion

Insertion Sort's median runtime rose sharply as the input size increased, from approximately 0.0015 ms at $n = 10$ to over 520 ms at $n = 20{,}000$, consistent with its $O(n^2)$ average-case time complexity. Randomized Quicksort grew more slowly, reaching roughly 4.5–4.6 ms at $n = 20{,}000$, consistent with expected $O(n \log n)$ performance. For Counting Sort, the key-range parameter had a strong effect because its running time is $O(n + k)$. With $k = n$, Counting Sort was fastest of the three algorithms for the larger inputs, taking approximately 1.92 ms at $n = 20{,}000$. With $k = 100n$, its median increased to about 78.62 ms at the same size, making Quicksort faster under that setting. Insertion Sort and Quicksort changed comparatively little between the two key ranges, since their costs depend primarily on input length and ordering rather than the maximum key value.